claude-windows / 73a3938e-1da3-4e28-bfa7-346a15718301

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\agent-a9c6b91377cc917c1.jsonl`
- SHA-256: `d73ce103695f53603b739c4a67f3eda690f8a5552041c29076e485b068c540de`
- Source modified: `2026-06-22T12:38:39+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_753ff171-b1d`
- session_id: `73a3938e-1da3-4e28-bfa7-346a15718301`

Transcript

1. user (2026-06-22T12:34:19.535Z)

You are adversarially reviewing the CODE in PR #319 of the local git repo (Khelsutra/badminton-highlight-indexer). The fix adds a helper `_label_clip_name` to `backend/worker/__main__.py` and uses it in `cmd_train` to derive a clip name from a label filename by stripping the `.rallies.csv`/`.csv` suffix AND a leading ISO-date prefix `^\d{4}-\d{2}-\d{2}_`, so that `labels/<YYYY-MM-DD>_<name>.rallies.csv` joins to `videos/<name>.<ext>`.

Read the actual changed code with: `git show origin/fix/worker-train-dated-label-join:backend/worker/__main__.py` (and `git diff origin/master..origin/fix/worker-train-dated-label-join -- backend/worker/__main__.py`). Do NOT trust my description ? read the code.

Verify, with concrete evidence:
1. `import re` is present at module level.
2. The regex `^\d{4}-\d{2}-\d{2}_` is anchored and only strips a genuine leading date prefix.
3. The downstream spec `name` (used in `specs.append({"name": name, ...})` and the synth trajectory filename) is the clean, date-stripped name on BOTH the detector and synth paths.
4. EDGE CASES ? adversarially hunt for any way the helper corrupts a legitimate name: e.g. a clip whose <name> itself begins with something date-like; a video object literally named `videos/2026-06-22_foo.mp4` (would its stem then fail to match a dated label?); double suffixes; empty result. State whether each is a real risk GIVEN the locked naming contract (labels/<date>_<name>, videos/<name>) or only theoretical.
5. Any regression vs the previous behavior for legacy bare/_proxy labels.

Return a verdict. Mark ok=false ONLY for a real defect that should block merge (not a theoretical edge case that the naming contract precludes ? list those as nits).

2. assistant (2026-06-22T12:34:21.930Z)

I'll review the actual code in PR #319. Let me start by reading the changed code.

3. user (2026-06-22T12:34:25.297Z)

```
From https://github.com/Khelsutra/badminton-highlight-indexer
 * branch            fix/worker-train-dated-label-join -> FETCH_HEAD
---DIFF---
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 081d8f5..08d8d84 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -20,6 +20,7 @@ import csv
 import json
 import logging
 import os
+import re
 import sys
```

```
 import tempfile
 from typing import Any, List
@@ -264,6 +265,29 @@ def cmd_infer(args: argparse.Namespace) -> int:
 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

+#: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
+#: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is
NOT
+#: part of the clip's join key, so it's stripped when deriving the videos/ stem.
+_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
+
+
+def _label_clip_name(fname: str) -> str:
+    """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label filename.
+
+    Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date prefix,
so a
+    canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
+
+    * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-stamped)
+    * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same stem)
+    * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of the
name, kept)
+    * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in <name>
survive)
+
+    Without the date strip, ``--trajectory-source detector`` can't match a dated label to its
bare
+    video stem, so every clip is skipped and train aborts with ``<2 videos`` (DATA_IN_GCS §7b
#1).
+    """
+    name = fname.replace(".rallies.csv", "").replace(".csv", "")
+    return _LABEL_DATE_PREFIX.sub("", name)
+

 def _probe_video_fps_width(path: str):
     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
@@ -380,8 +404,8 @@ def cmd_train(args: argparse.Namespace) -> int:
     print(f"[worker] trajectory source: {args.trajectory_source}")
     specs: List[dict] = []
     for uri in label_uris:
-        fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
-        name = fname.replace(".rallies.csv", "").replace(".csv", "")
+        fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
+        name = _label_clip_name(fname)                         # -> GX020094 (date prefix
stripped)
         local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
         if args.trajectory_source == "detector":
             vuri = video_by_stem.get(name)
```

4. assistant (2026-06-22T12:34:28.547Z)

Let me see the full context around `cmd_train`, especially the synth path and how `name` flows downstream.

5. user (2026-06-22T12:34:29.575Z)

```
        runner = _resolve_train_detector(args, config)
        if runner is None:
            return 2
        for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
```

```
            vname = storage.parse_uri(vuri).name
            if not vname.lower().endswith(_VIDEO_EXTS):
                continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow the video
            video_by_stem[os.path.splitext(vname)[0]] = vuri

    print(f"[worker] trajectory source: {args.trajectory_source}")
    specs: List[dict] = []
    for uri in label_uris:
        fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
        name = _label_clip_name(fname)                         # -> GX020094 (date prefix
stripped)
        local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
        if args.trajectory_source == "detector":
            vuri = video_by_stem.get(name)
            if not vuri:
                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
                continue
            local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                        config=storage_cfg)
            video_id = compute_video_id(local_video)
            # Real fps/width drive the trajectory frame->time mapping; PROBE (don't guess) ? a
            # wrong fps silently mis-aligns every label, so skip the video if ffprobe can't read
it.
            meta = _probe_video_fps_width(local_video)
            if meta is None:
                print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ? skipping
(won't guess).")
                continue
            fps, frame_width = meta
            traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
            if not traj:
                print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
                continue
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": fps, "frame_width": frame_width})
        else:
            from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
            traj = os.path.join(traj_dir, f"{name}_traj.csv")
            synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": args.fps, "frame_width": args.frame_width})

    if len(specs) < 2:
        print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
        return 2
    manifest_path = os.path.join(scratch, "manifest.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(specs, f, indent=2)
    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                       if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
```

```
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="python -m backend.worker",
                                description="Storage-aware worker: pull ? run pipeline ? push.")
    p.add_argument("-v", "--verbose", action="store_true",
                   help="DEBUG logging (the native detector command, frame counts, per-stage
detail)")
    sub = p.add_subparsers(dest="cmd", required=True)
```

6. assistant (2026-06-22T12:34:32.949Z)

Now let me verify the `import re` at module level and check the broader context around the imports, and confirm `name` isn't used elsewhere (e.g., for the local label path).

7. user (2026-06-22T12:34:35.554Z)

```
"""
from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
===GREP name usage in cmd_train===
23:import re
188:    db = Database(os.path.join(scratch, config.output.db_name))
199:                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
201:        seg_name = args.segmenter or config.indexing.default_segmenter
202:        segmenter_cls = segmenter_registry.get(seg_name)
204:            print(f"[worker] unknown segmenter: {seg_name!r} "
207:        segmenter_actual = seg_name
268:#: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
274:def _label_clip_name(fname: str) -> str:
```

```
275:    """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label filename.
282:    * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of the
name, kept)
283:    * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in <name>
survive)
288:    name = fname.replace(".rallies.csv", "").replace(".csv", "")
289:    return _LABEL_DATE_PREFIX.sub("", name)
399:            vname = storage.parse_uri(vuri).name
400:            if not vname.lower().endswith(_VIDEO_EXTS):
402:            video_by_stem[os.path.splitext(vname)[0]] = vuri
407:        fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
408:        name = _label_clip_name(fname)                         # -> GX020094 (date prefix
stripped)
409:        local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
411:            vuri = video_by_stem.get(name)
413:                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
415:            local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
422:                print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ? skipping
(won't guess).")
427:                print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
429:            specs.append({"name": name, "trajectory": traj, "labels": local_label,
433:            traj = os.path.join(traj_dir, f"{name}_traj.csv")
435:            specs.append({"name": name, "trajectory": traj, "labels": local_label,
529:        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
```

8. assistant (2026-06-22T12:34:40.629Z)

Now let me verify the key edge case the prompt raised: the local_label path uses `fname` (full filename with date), while `name` (stripped) is only used for join key and synth trajectory filename. Let me confirm `local_label` uses `fname` and check there's no collision risk. Let me also check the naming contract in the docs and look for tests.

9. user (2026-06-22T12:34:43.041Z)

```
=== local_label uses fname (full name w/ date) ? good for fetch path uniqueness ===
fname goes to labels_dir, name goes to traj filename + join key

=== Check tests for _label_clip_name ===
commit 56c1415299e40f90c7d8d3ef3825b73a50a82f77
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Mon Jun 22 18:00:22 2026 +0530

    style(worker-test): drop unused os import (ruff F401)

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

 tests/test_worker.py | 1 -
 1 file changed, 1 deletion(-)
```

10. user (2026-06-22T12:34:46.522Z)

```
56c1415 style(worker-test): drop unused os import (ruff F401)
b3b4952 test(worker): cover cmd_train per-label join/skip branches
a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train
```

11. assistant (2026-06-22T12:34:48.922Z)

Good - there are tests. Let me confirm the branch contains only its own commits (no sibling contamination) and read the test coverage.

12. user (2026-06-22T12:34:50.387Z)

```
4:segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
20:def test_coerce(raw, expected):
24:def test_apply_overrides_on_typed_config():
26:    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
28:    assert cfg.indexing.skip_ai_handoff is True
33:def test_apply_overrides_rejects_bad_format():
39:def test_write_intervals_csv(tmp_path):
50:def test_write_report_json(tmp_path):
58:def test_parser_infer_accepts_set_and_uris():
61:        "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
63:    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
111:def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
122:        "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
143:def test_setup_logging_levels():
151:def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
171:    msgs = " ".join(r.getMessage() for r in caplog.records)
178:def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
195:    ("2026-06-22_GX020094.rallies.csv", "GX020094"),            # canonical dated label
(DATA_IN_GCS §7.1)
196:    ("GX020094.rallies.csv", "GX020094"),                       # legacy bare label -> SAME
clip stem
197:    ("GX020094_proxy.rallies.csv", "GX020094_proxy"),           # _proxy is part of the
name, kept
198:    ("2026-06-21_mahadevpura_1.rallies.csv", "mahadevpura_1"),  # underscores in <name>
survive
200:    ("2026-06-22_GX020094.csv", "GX020094"),                    # dated + bare .csv
202:def test_label_clip_name_strips_date_prefix(fname, expected):
203:    """A dated label and a bare label must both resolve to the bare videos/<name> stem."""
204:    assert wmain._label_clip_name(fname) == expected
207:def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
208:    """Regression (DATA_IN_GCS §7b blocker #1): canonical labels carry an ISO-date prefix
209:    (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The label-
derived
210:    clip name must strip the date so the video join succeeds ? otherwise every clip is
skipped and
224:    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n")
225:    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header +
"1,3.0,6.0,winner,badminton,\n")
226:    # ... joining to BARE-stem videos
237:        bdir = os.path.join(out, ver)
240:            with open(os.path.join(bdir, fn), "w") as f:
248:    assert rc == 0  # before the fix: every clip skipped -> rc 2 ("<2 videos")
250:    assert len(specs) == 2                                      # both dated labels
joined
251:    assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"}  # downstream name is
date-stripped
254:def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
255:    """A dated label whose <name> has no videos/<name>.<ext> is SKIPPED (the join-miss
branch),
256:    while the labels that DO join are kept ? so one orphan label can't abort an otherwise-
valid run."""
269:    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)
270:    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header + row)
271:    (labels_dir / "2026-06-20_orphan.rallies.csv").write_text(header + row)  # NO
videos/orphan.*
282:        bdir = os.path.join(out, ver)
285:            with open(os.path.join(bdir, fn), "w") as f:
293:    assert rc == 0  # the orphan label was skipped; the two joinable clips carried the run
298:def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(tmp_path, monkeypatch):
312:        (labels_dir / f"2026-06-22_{stem}.rallies.csv").write_text(header + row)
315:    # probe fails ONLY for badfps -> that clip is skipped (a wrong fps mis-aligns every
label).
321:        # writes a real trajectory for everything EXCEPT notraj (empty result -> skipped).
```

13. assistant (2026-06-22T12:34:55.612Z)

The test coverage is thorough. Now let me run the test suite for the worker to verify everything is actually green, and run ruff on the changed file.

14. user (2026-06-22T12:34:57.126Z)

```
=== current branch ===
docs/data-gcs-train-blocker-resolved
```

15. assistant (2026-06-22T12:35:01.655Z)

Let me check out the PR branch in a clean way to run its tests. First confirm working tree state, then run the worker tests against the PR branch.

16. user (2026-06-22T12:35:02.603Z)

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
=== HEAD ===
62bcdf4 docs(data): mark DATA_IN_GCS §7b blocker #1 resolved by #319
```

17. assistant (2026-06-22T12:35:06.245Z)

Working tree is clean (only untracked artifacts/ and a doc). Let me run the worker tests against the PR branch code. I'll use a worktree-free approach: check out the PR branch's test + source files into a temp run. Actually, the cleanest is to run pytest with the PR branch checked out. Let me create a detached checkout.

18. user (2026-06-22T12:35:09.423Z)

```
Preparing worktree (detached HEAD 56c1415)
HEAD is now at 56c1415 style(worker-test): drop unused os import (ruff F401)
```

19. user (2026-06-22T12:35:17.069Z)

```
.............................                                            [100%]
29 passed in 2.28s
```

20. assistant (2026-06-22T12:35:19.598Z)

All 29 worker tests pass. Now let me run ruff and mypy on the changed file, and run the full test suite to confirm no regressions.

21. user (2026-06-22T12:35:48.205Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 1 source file
```

22. assistant (2026-06-22T12:35:50.887Z)

Ruff clean, mypy clean (only informational notes). Now the full test suite to confirm no regressions.

23. user (2026-06-22T12:37:08.010Z)

```
========================================================================
.....................ss.............................................. [  5%]
........................................................................ [ 10%]
........................................................................ [ 15%]
........................................................................ [ 20%]
........................................................................ [ 25%]
........................................................................ [ 30%]
........................................................................ [ 35%]
........................................................................ [ 40%]
........................................................................ [ 45%]
........................................................................ [ 50%]
........................................................................ [ 55%]
........................................................................ [ 60%]
........................................................................ [ 65%]
........................................................................ [ 70%]
.....................................................sssss.............. [ 75%]
........................................................................ [ 80%]
........................................................................ [ 85%]
........................................................................ [ 90%]
........................................................................ [ 95%]
.........................................................            [100%]
1423 passed, 7 skipped in 73.58s (0:01:13)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

24. assistant (2026-06-22T12:37:10.721Z)

Full suite green (1423 passed, 7 skipped). Now let me adversarially probe the helper's edge cases directly with the actual code, and verify the naming contract in the docs.

25. user (2026-06-22T12:37:18.361Z)

```
'2026-06-22_GX020094.rallies.csv' -> 'GX020094'
'GX020094.rallies.csv' -> 'GX020094'
'GX020094_proxy.rallies.csv' -> 'GX020094_proxy'
'2026-06-21_mahadevpura_1.rallies.csv' -> 'mahadevpura_1'
'2026-06-22_2026-06-22_foo.rallies.csv' -> '2026-06-22_foo'
'2026-06-22_.rallies.csv' -> ''
'.rallies.csv' -> ''
'2026-13-99_foo.rallies.csv' -> 'foo'
'20260622_foo.rallies.csv' -> '20260622_foo'
'2026-06-22-foo.rallies.csv' -> '2026-06-22-foo'
'foo_2026-06-22_bar.rallies.csv' -> 'foo_2026-06-22_bar'
'match.rallies.csv.rallies.csv' -> 'match'
'GX_2026-06-22.rallies.csv' -> 'GX_2026-06-22'
'2026-06-22_GX020094.csv' -> 'GX020094'
```

26. assistant (2026-06-22T12:37:25.760Z)

The probe confirms the regex is properly anchored (`^`) and only strips a genuine leading `YYYY-MM-DD_`. Now the critical adversarial question from the prompt: **a video literally named `videos/2026-06-22_foo.mp4`**. The join works as `video_by_stem.get(name)` where `video_by_stem` is keyed by the bare video stem. Let me verify how the video stem is computed ? it uses `os.path.splitext(vname)[0]`, with NO date stripping. So if a video itself has a date-like stem, the label-derived `name` (date-stripped) would NOT match. Let me confirm the naming contract in the docs precludes this.

27. user (2026-06-22T12:37:28.613Z)

```
=== §7 naming contract ===
```

28. user (2026-06-22T12:37:32.147Z)

```
backend/storage/gcs.py
=== search docs for the contract ===
fatal: path 'docs/DATA_IN_GCS.md' exists on disk, but not in 'origin/fix/worker-train-dated-
label-join'
```

29. assistant (2026-06-22T12:37:34.974Z)

```
The doc isn't on the PR branch (it's local/untracked or on another branch). Let me find it.
```

30. user (2026-06-22T12:37:37.523Z)

```
31:`s3://`, `gdrive://`, or a bare local path. Pick the backend with **config + creds, no code
change**.
42:| `videos/` | source videos / proxies (`<clip>.mp4`) | **input** to `worker infer` & `worker
train` |
43:| `labels/` | golden rally labels (`<clip>.rallies.csv`) | **input** ground truth to `worker
train` + all eval |
51:> *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
**synthetically
53:> detector` (needs `videos/`). A `manifest.json` only appears as an **output** inside each
62:> **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
`labels/<date>_<name>.rallies.csv`
63:> (= original-6 + `GX010137`/`GX010141` + 7 new; the 7 old `_proxy`/bare dupes were deleted),
and
69:| `labels/` | 7 files: `GX010128`, `mahadevpura-1`, + `Badminton BXH 2_proxy`, `Boxhill
Doubles_proxy`, `GX020094_proxy`, `GX030094_proxy`, `mahadevpura-2_proxy` | ? **inconsistent
naming** (`_proxy` on some, not others); ? **missing** the original-6 labels + `GX010137`,
`GX010141` |
70:| `videos/` | 3: `GX030094_proxy.mp4`, `mahadevpura-1.MP4`, `mahadevpura_smoke_180s.mp4` | ?
corpus videos/proxies almost entirely absent |
78:`weights/`) and **inconsistent `labels/` naming**, and is **missing the bulk of the corpus
inputs**
79:(`videos/`, all `trajectories/`, most `labels/`). It reflects past *dry runs*, not a complete
corpus.
94:  --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
97:# TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds manifest
on-box,
104:A bare/relative path resolves under a configured `input_root`/`input_backend`
132:naming decision. Sketch:
137:# labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
141:**Phase 2 ? videos ? `videos/<name>.mp4`** (bucket name = the eval `name`, consistent with
142:`trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
151:  `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
155:  -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses local
bandwidth.
159:? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
160:are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-dupe-
then-delete
176:1. **`labels/` naming ? LOCKED (owner, 2026-06-22):** `labels/<YYYY-MM-
DD>_<name>.rallies.csv`, where
177:   `<name>` is the eval/manifest clip name and `<YYYY-MM-DD>` is the label's **authoring
date** =
179:   a restored C: copy shows today, the F: archive keeps the real date). Fixes the `_proxy`
inconsistency
180:   and disambiguates recycled GoPro names by date. *(Future, when videos are in the bucket:
optionally
182:2. **Cache `trajectories/` ? YES, done 2026-06-22.** `trajectories/<name>.csv` cached so
cloud/CI eval +
191:> authoring date, build the `gs://` manifest) ? `gcloud storage cp` (planner kept in
gitignored
194:> bare `GX010128`/`mahadevpura-1`) were confirmed **byte-identical (MD5)** to the new
canonical names
```

```
195:> (content also on C:/F:) and **deleted** ? `labels/` now holds exactly the 15 canonical
objects.
207:   ? dated labels broke `worker train --trajectory-source detector`.** *History:*
`cmd_train` derived the
209:   `video_by_stem.get(name)` against bare video stems (`GX020094`), so every clip was
skipped and train
212:   `.rallies.csv`/`.csv` suffix AND a leading `^\d{4}-\d{2}-\d{2}_` prefix, so a dated label
and a legacy
213:   bare label both resolve to the bare `videos/<name>` stem (preserves the locked human-
readable naming).
214:   Tested: parametrized helper cases + a dated-label?bare-video `cmd_train` join regression
+ per-label
236:| Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
238:| Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified dupes,
deleted) |
239:| Blocker §7b#1: dated labels join in `worker train --trajectory-source detector` | ?
2026-06-22 ([#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)) ?
`_label_clip_name` strips the ISO-date prefix |
240:| Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done; **1/15**
demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the owner-box
script |
=== file on disk? ===
-rw-r--r-- 1 avidu 197609 17395 Jun 22 18:02 C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/DATA_IN_GCS.md
EXISTS
```

31. assistant (2026-06-22T12:37:47.332Z)

This confirms the locked naming contract (line 176-179): `labels/<YYYY-MM-DD>_<name>.rallies.csv` where `<name>` is the eval/manifest clip name, and **`videos/<name>.<ext>`** (line 141: "bucket name = the eval `name`"). The video objects are bare `<name>`, NOT date-prefixed.

So the adversarial case "video literally named `videos/2026-06-22_foo.mp4`" would require the eval `<name>` itself to be `2026-06-22_foo` ? which would mean the label is `labels/<date>_2026-06-22_foo.rallies.csv`. The contract puts the authoring date as the prefix and `<name>` is the clip name. Under the contract, video stems are bare names and would never themselves be a date prefix because that's reserved by convention. This is the double-date case in my probe ? only the *outer* (leading) date is stripped, correctly leaving `2026-06-22_foo`, which would match a video `videos/2026-06-22_foo.mp4`. So even that pathological case round-trips correctly.

Let me verify the legacy `_proxy`/bare regression behavior one more time and check the old vs new code path divergence on the synth trajectory filename collision risk.

32. user (2026-06-22T12:37:52.967Z)

```
LEGACY (no-date) regression check: new==old ?
  'GX020094_proxy.rallies.csv': old='GX020094_proxy' new='GX020094_proxy' same=True
  'GX010128.rallies.csv': old='GX010128' new='GX010128' same=True
  'mahadevpura-1.rallies.csv': old='mahadevpura-1' new='mahadevpura-1' same=True
  'Boxhill Doubles_proxy.rallies.csv': old='Boxhill Doubles_proxy' new='Boxhill Doubles_proxy'
same=True
  'mahadevpura_smoke_180s.rallies.csv': old='mahadevpura_smoke_180s'
new='mahadevpura_smoke_180s' same=True
```

33. assistant (2026-06-22T12:37:58.614Z)

Confirmed: for any label WITHOUT a leading date prefix, `new() == old()` exactly. Zero regression for legacy bare/`_proxy` labels. The change is purely additive: it only strips when a genuine `^YYYY-MM-DD_` prefix is present.

Now let me clean up the worktree.

34. user (2026-06-22T12:38:00.424Z)

cleaned

35. assistant (2026-06-22T12:38:15.113Z)

I have everything I need. Let me consolidate the findings.

Summary of verification:

1. **`import re` at module level** ? Confirmed present at line 23 of `backend/worker/__main__.py`, alongside other stdlib imports.

2. **Regex anchored & strips only genuine date prefix** ? `_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")` is `^`-anchored. Probe confirmed: `foo_2026-06-22_bar` (date not leading) ? NOT stripped; `2026-06-22-foo` (dash not underscore) ? NOT stripped; `20260622_foo` (no dashes) ? NOT stripped.

3. **Downstream `name` is the clean stripped name on BOTH paths** ? `name = _label_clip_name(fname)` (line 408) feeds: detector path join `video_by_stem.get(name)` (411), `specs.append({"name": name, ...})` (429); synth path traj filename `f"{name}_traj.csv"` (433) and `specs.append({"name": name, ...})` (435). The `local_label` fetch path correctly uses the *full* `fname` (with date) (line 409/labels_dir), so two same-clip labels on different dates can't clobber each other's local file ? good.

4. **Edge cases** ? All theoretical-only under the locked contract (`labels/<YYYY-MM-DD>_<name>`, `videos/<name>`, owner-LOCKED 2026-06-22, DATA_IN_GCS §7.1):
   - Double-date `2026-06-22_2026-06-22_foo` ? strips only the outer/leading date ? `2026-06-22_foo`. This is actually *correct*: if a clip's eval `<name>` were literally `2026-06-22_foo`, its video object is `videos/2026-06-22_foo.mp4` and the join still matches. Round-trips correctly.
   - Video literally `videos/2026-06-22_foo.mp4`: video stems are NOT date-stripped (`os.path.splitext(vname)[0]`), but under the contract the matching label would be `labels/<authoring-date>_2026-06-22_foo.rallies.csv`, and the helper strips only the leading authoring-date ? `2026-06-22_foo` ? matches. Not a real risk.
   - Empty result (`2026-06-22_.rallies.csv` ? `""`): contract guarantees a non-empty `<name>`; an empty stem simply misses the join and is skipped (no crash). Theoretical only.
   - Invalid date like `2026-13-99_foo`: regex matches structurally, strips ? `foo`. Harmless and not producible under the contract.

5. **No regression for legacy bare/`_proxy` labels** ? Proven: for every label without a leading date prefix, `new() == old()` byte-for-byte (`GX020094_proxy`, `GX010128`, `mahadevpura-1`, `Boxhill Doubles_proxy`, etc.). Purely additive.

Verification run: worker tests 29 passed; full suite `run_all_tests.py` ? 1423 passed / 7 skipped; `ruff check` clean; `mypy` clean. Tests include parametrized helper cases + dated-label?bare-video join regression + orphan-skip + unprobeable/trajless-skip branches.

36. user (2026-06-22T12:38:29.274Z)

Structured output provided successfully

37. assistant (2026-06-22T12:38:39.374Z)

Verdict: **ok=true** ? no blocking defects.

The `_label_clip_name` fix in PR #319 is correct and safe to merge. The regex is properly `^`-anchored and strips only a genuine leading `YYYY-MM-DD_` prefix; the clean date-stripped `name` flows consistently through both the detector and synth paths (join key, `specs.append({"name": ...})`, and synth trajectory filename), while the on-disk label fetch correctly keeps the full dated `fname` so same-clip labels on different dates can't clobber each other. There is zero regression for legacy bare/`_proxy` labels (proven byte-for-byte `new()==old()` whenever no date prefix is present ? the change is purely additive). Every adversarial edge case (double-date, date-like video stem, empty result, invalid calendar date)

is either correct-by-construction or precluded by the owner-locked naming contract (`labels/<date>_<name>`, `videos/<name>`), so they're nits, not blockers.

Verified locally on a clean detached worktree: 29 worker tests pass; full suite 1423 passed / 7 skipped; `ruff` and `mypy` clean. Test coverage is thorough (parametrized helper cases + dated?bare join regression + orphan-skip + unprobeable/trajless-skip branches).

Relevant file: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py` (helper at lines 268-289, `cmd_train` use at lines 407-435); tests at `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py` (lines 195-330); naming contract at `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA_IN_GCS.md` (lines 176-179, 141).